Segment-8

Stream:

In C++, a stream refers to a sequence of characters that are transferred between the program and input/output (I/O) devices.

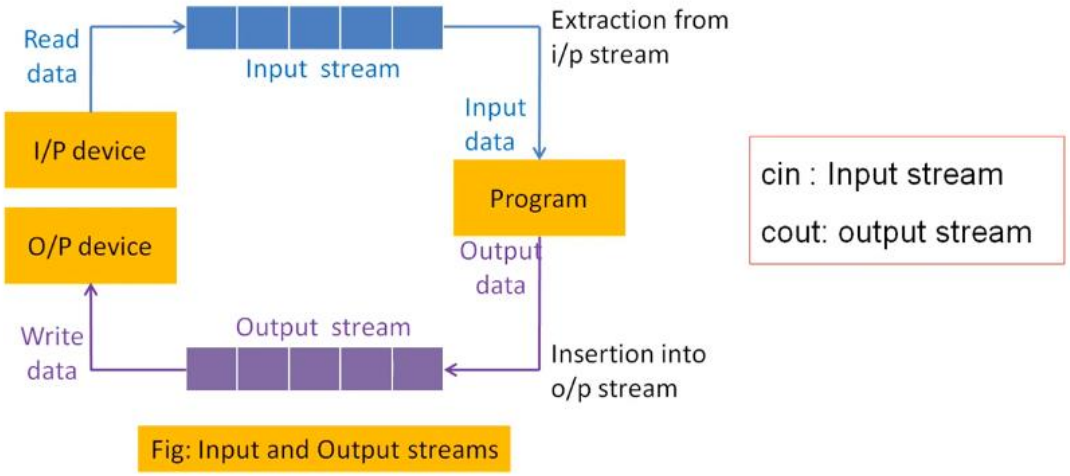

Fig: Input and Output streams

Stream classes:

Stream classes in C++ are used to input and output operations on files and I/O devices. These classes have specific features and to handle input and output of the program.

- **ios class** − This class is responsible for handling both input and output stream as both **istream class** and o**stream class** is inherited into it. It provides function of both **istream class** and o**stream class** for handling chars, strings and objects such as **get, getline, read, ignore, putback, put, write** etc.
- **istream Class** − The istream class handles the input stream in c++ programming language. These input stream objects are used to read and interpret the input as a sequence of characters. The cin handles the input.
- **ostream class** − The ostream class handles the output stream in c++ programming language. These output stream objects are used to write data as a sequence of characters on the screen. cout and puts handle the output streams in c++ programming language.
- **Ifstream classs-** An ifstream object represents an input file stream, which is used to read data from a file
- **Ofstream class-** An Ofstream object represents an output file stream, which is used to create files and write information to the files.

Formatted I/O:

C++ helps you to format the I/O operations like determining the number of digits to be displayed after the decimal point, specifying number base etc.

Example:

- If we want to add + sign as the prefix of output, we can use the formatting to do so:

stream.setf(ios::showpos)

If input=100, output will be +100

- If we want to add trailing zeros in output to be shown when needed using the formatting:

stream.setf(ios::showpoint)

If input=100.0, output will be 100.000

NB. To set a format flag, use the **setf()** function.

There are two ways to format the data
1. Using the ios class or various ios member functions.
2. Using manipulators(special functions)

Formatting using the ios members:
The stream has the format flags that control the way of formatting. it means Using this setf function, we can set the flags, which allow us to display a value in a particular format.

Few standard ios class functions are:

- **width():** The width method is used to set the required field width. The output will be displayed in the given width
- **precision():** The precision method is used to set the number of the decimal point to a float value
- **fill():** The fill method is used to set a character to fill in the blank space of a field
- **setf():** The setf method is used to set various flags for formatting output
- **unsetf():** The unsetf method is used To remove the flag setting

Example:

```
#include<bits/stdc++.h>
using namespace std;
void IOS_width()
{
        cout << "Implementing width\n\n";
        char c = 'A';
        cout.width(5);
        cout << c <<"\n";
        int temp = 10;
        cout<<temp;
}
void IOS_precision()
{
 cout << "Implementing precision\n\n";
        cout.precision(2);
        cout<<3.1422;
}
// The fill() function fills the unused white spaces in a value  with a character of choice.
void IOS_fill()
{
```

```
	cout << "Implementing fill\n\n";
	char ch = 'a';
	cout.fill('*');
	cout.width(10);
	cout<<ch <<"\n";
	int i = 1;
	cout.width(5);
	cout<<i;
}
void IOS_setf()
{
	cout << "Implementing setf\n\n";
	int val1=100, val2=200;
	cout.setf(ios::showpos);
	cout<<val1<<" "<<val2;
}
void IOS_unsetf()
{
	cout << "Implementing unsetf\n\n";
	cout.setf(ios::showpos|ios::showpoint);
	// Clear the showflag flag without affecting the showpoint flag
	cout.unsetf(ios::showpos);
	cout<<200.0;
}



// Driver Method
int main()
{
	IOS_width();
	IOS_precision;
	IOS_fill();
	IOS_setf();
	IOS_unsetf();
	return 0;
}
```

Output:

Implementing width

A

10

Implementing precision

3.12

Implementing fill

*********a

****1

Implementing setf

+100 +200

Implementing unsetf

200.000

Formatting using Manipulators:
The second way to format parameters of a stream is through the use of special functions called manipulators that can be included in an I/O expression.
The standard manipulators are shown below:

- boolalpha: The boolalpha manipulator of stream manipulators in C++ is used to turn on bool alpha flag
- **dec:** The dec manipulator of stream manipulators in C++ is used to turn on the dec flag
- **endl:** The endl manipulator of stream manipulators in C++ is used to Output a newline character.
- **and:** The and manipulator of stream manipulators in C++ is used to Flush the stream
- **ends:** The ends manipulator of stream manipulators in C++ is used to Output a null
- **fixed:** The fixed manipulator of stream manipulators in C++ is used to Turns on the fixed flag
- **hex:** The hex manipulator of stream manipulators in C++ is used to Turns on hex flag
- **internal**: The internal manipulator of stream manipulators in C++ is used to Turns on internal flag
- **left**: The left manipulator of stream manipulators in C++ is used to Turns on the left flag
- noboolalpha: The noboolalpha manipulator of stream manipulators in C++ is used to Turns off bool alpha flag
- noshowbase: The noshowbase manipulator of stream manipulators in C++ is used to Turns off showcase flag
- noshowpoint: The noshowpoint manipulator of stream manipulators in C++ is used to Turns off show point flag
- **noshowpos**: The noshowpos manipulator of stream manipulators in C++ is used to Turns off showpos flag
- noskipws: The noskipws manipulator of stream manipulators in C++ is used to Turns off skipws flag
- nouppercase: The nouppercase manipulator of stream manipulators in C++ is used to Turns off the uppercase flag

For more formatted I/O see-> https://www.geeksforgeeks.org/formatted-i-o-in-c/

Stream Inserter Operator <<

The Stream Inserters are functions used to insert data or objects into an output source.The insertion operator for user-defined types must perform two levels of translations: It first breaks down the user-defined type into built-in datatypes, and then converts the built-in datatypes to a generic stream of bytes directed to some output device.

The general form of Inserter:

```
ostream & operator < <( ostream &stream , class_name obj )
{
// body of inserter
   return stream ;
}
```

Creating own inserters

```
// Use a friend inserter for objects of type coord.
#include <iostream>
using namespace std;

class coord {
  int x, y;
public:
  coord() { x = 0; y = 0; }
  coord(int i, int j) { x = i; y = j; }
  friend ostream &operator<<(ostream &stream, coord ob);
};

ostream &operator<<(ostream &stream, coord cb)
{
  stream << ob.x << ", " << ob.y << '\n';
  return stream;
}

int main()
{
  coord a(1, 1), b(10, 23);


   cout << a << b;

   return 0;
}
```

This program displays the following:

```
1, 1
10, 23
```

Stream Extractors Operator >>

Extractors are functions used to extract or remove data or objects from an input source. The right bit Shift operator (>>) is overloaded to extract data from an input stream and then assigns it to built-in or user-defined data types.

Creating own extractor

```
#include <iostream>
using namespace std;

class coord {
  int x, y;
public:
  coord() { x = 0; y = 0; }
  coord(int i, int j) { x = i; y = j; }
  friend ostream &operator<<(ostream &stream, coord ob);
  friend istream &operator>>(istream &stream, coord &ob);
};

ostream &operator<<(ostream &stream, coord ob)
{
  stream << ob.x << ", " << ob.y << '\n';
  return stream;
}

istream &operator>>(istream &stream, coord &ob)
{
  cout << "Enter coordinates: ";
  stream >> ob.x >> ob.y;
  return stream;
}

int main()
{
  coord a(1, 1), b(10, 23);

    cout << a << b;

    cin >> a;
    cout << a;

    return 0;
}
```

File Handling:

File handling in C++ is a mechanism to store the output of a program in a file and help perform various operations on it. Files help store these data permanently on a storage device.

Opening a File

A file must be opened before you can read from it or write to it. Either ofstream or fstream object may be used to open a file for writing. And ifstream object is used to open a file for reading purpose only.

Sr.No	Mode Flag & Description
1	ios::app Append mode. All output to that file to be appended to the end.
2	ios::ate

	Open a file for output and move the read/write control to the end of the file.
3	ios::in Open a file for reading.
4	ios::out Open a file for writing.
5	ios::trunc If the file already exists, its contents will be truncated before opening the file.

File Operations in C++

C++ provides us with four different operations for file handling. They are:

1. **open()** – This is used to create a file.
2. **read() –** This is used to read the data from the file.
3. **write() –** This is used to write new data to file.
4. **close() –** This is used to close the file.

Creating File and write data in the file:

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
int main(){
   string name;
   ofstream FileName;
   FileName.open("FileName.txt",ios::out|ios::app);
   FileName<<"IIUC\n";

   cout<<" Enter Your Name: \n";
   getline(cin,name);
   FileName<<name;
   FileName.close();
   return 0;
}
```

Program for Reading from File:

```
#include<iostream>
#include<fstream>
#include<string>
using namespace std;
int main(){
   string name;
   ifstream file("FileName.txt");
   while(getline(file,name)){
         cout<<name<<endl;
   }

   file.close();
   return 0;
}
```

Previous Question Solve:

5 a)	Write stream classes hierarchy for console I/O operations. **Or,** Formulate the difference between manipulators and **ios** member functions.	3
b)	Write a program to generate a file named "Numbers.txt" that contains 50 floating-point numbers between 0 and 1. Then, read the contents of the file and display them on the screen. Implement the program, generate the file with the specified numbers, read the contents from the file, and display them on the screen. Please provide the complete program and the displayed numbers on the screen.	4
c)	Write a program that implements following functions: i) Width() ii) Precision() iii) Fill()	3

1) What is manipulator? Formulate the differences between manipulators and ios member functions.

Ans=>

In the context of C++ programming, a manipulator is a function that manipulates the formatting or state of an input or output stream. These manipulators are typically used with the std::cout, std::cin.

Manipulators are standalone functions that take a stream object as input and return a modified version of the stream object. For example, the std::endl manipulator adds a newline character to the stream and flushes the buffer. Another example is the std::setw() manipulator, which sets the width of the next output field.

The main differences between manipulators and I/O member functions are:

- Manipulators are standalone functions, while I/O member functions are member functions of the stream classes.
- Manipulators modify the formatting or state of the stream, while I/O member functions perform I/O operations on the stream data.
- Manipulators are typically used with the insertion (<<) and extraction (>>) operators, while I/O member functions are called explicitly on the stream object.

2) Write a program that implements the following ios functions: width(), precision(0), fill, setf() Write the output of the program.
Ans=>

```
#include <iostream>
#include <iomanip>

int main() {
   int num = 12345;
   double pi = 3.14159;

   std::cout << "Using width() function:" << std::endl;
   std::cout.width(10);
   std::cout << num << std::endl;

   std::cout << "Using precision() function:" << std::endl;
   std::cout.precision(3);
   std::cout << pi << std::endl;

   std::cout << "Using fill() function:" << std::endl;
   std::cout.width(10);
   std::cout.fill('*');
   std::cout << num << std::endl;

   std::cout << "Using setf() function:" << std::endl;
   std::cout.setf(std::ios::showpoint);
   std::cout.setf(std::ios::fixed);
   std::cout.precision(2);
   std::cout << pi << std::endl;
   return 0;
}
```

Output:

```
Using width() function:
     12345
Using precision() function:
3.14
Using fill() function:
*****12345
Using setf() function:
3.14
```

3) Design a program to write the following information to a file called WhoAre You.txt :
Name: xxxxxxx
Semester: Spring 2022
Course Code: CSE-1221
Course Title: Computer Programming 2
Ans=>

```
#include <iostream>
```

```
#include <fstream>

int main() {
   std::ofstream outfile("Who Are You.txt");

   if (outfile.is_open()) {
      outfile << "Name: xxxxxxx\n";
      outfile << "Semester: Spring 2022\n";
      outfile << "Course Code: CSE-1221\n";
      outfile << "Course Title: Computer Programming 2\n";

      outfile.close();
      std::cout << "File written successfully.\n";
   } else {
      std::cout << "Error opening file.\n";
   }
   return 0;
}
```

4) Design an inserter for a class named VECTOR which has two private integer members named x and y.

Ans=>

Here's an example implementation of an inserter for a class named VECTOR with two private integer members x and y:

```
#include <iostream>

class VECTOR {
private:
   int x;
   int y;

public:
   VECTOR(int x, int y) : x(x), y(y) {}

   friend std::ostream& operator<<(std::ostream& os, const VECTOR& vec);
};

std::ostream& operator<<(std::ostream& os, const VECTOR& vec) {
   os << "(" << vec.x << ", " << vec.y << ")";
   return os;
}
int main() {
   VECTOR v(3, 4);
   std::cout << "Vector v: " << v << std::endl;
   return 0;
}
```

5) Design an extractor for a class named VECTOR which has two private integer members named x and y.

Ans=>
Here's an example implementation of an extractor for a class named VECTOR with two private integer members x and y:

```
#include <iostream>
#include <sstream>

class VECTOR {
private:
    int x;
    int y;

public:
    VECTOR(int x, int y) : x(x), y(y) {}

    friend std::istream& operator>>(std::istream& is, VECTOR& vec);
};

std::istream& operator>>(std::istream& is, VECTOR& vec) {
    // Read input string from stream
    std::string input;
    std::getline(is, input);

    // Parse input string into x and y components
    std::istringstream iss(input);
    char c1, c2, c3;
    int x, y;
    iss >> c1 >> x >> c2 >> y >> c3;

    // Check if parsing was successful
    if (iss.fail() || c1 != '(' || c2 != ',' || c3 != ')') {
        is.setstate(std::ios::failbit);
        return is;
    }

    // Set x and y components of vector
    vec.x = x;
    vec.y = y;

    return is;
}
int main() {
    VECTOR v(0, 0);
    std::cout << "Enter vector components (x,y): ";
    std::cin >> v;
    std::cout << "Vector v: (" << v.getX() << ", " << v.getY() << ")\n";
    return 0;
}
```

6)

Write a program to create a file named "templeRun.txt". If the file does not exist, store an integer taken from the keyboard. This integer refers to the high score. If the file does exist then read the integer it contains and show output:
Highscore: XXXXX
where xxxxx denotes the value templeRun.txt contains.
Ans=>

```
#include <iostream>
#include <fstream>

int main() {
   std::ifstream inFile("templeRun.txt");
   int highScore;

   if (!inFile.is_open()) {
      // File does not exist, so get the high score from the user
      std::cout << "Enter your high score: ";
      std::cin >> highScore;

      std::ofstream outFile("templeRun.txt");
      outFile << highScore;
      outFile.close();
   } else {
      // File exists, so read the high score from the file
      inFile >> highScore;
      inFile.close();

      std::cout << "High score: " << highScore << std::endl;
   }

   return 0;
}
```

Muhammad Nazim Uddin
Lecturer(Adjunct)
Dept of CSE,IIUC